

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-xxxx-xxxx

**Rozšířená šablóna záverečnej práce na FEI STU
v Bratislave v systéme $\text{\LaTeX}$**

Bakalárska práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-xxxx-xxxx

**Rozšířená šablóna záverečnej práce na FEI STU
v Bratislave v systéme $\text{\LaTeX}$**

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav multimediálnych informačných a komunikačných technológií
Školiteľ:	Ing. Marek Vančo, PhD.

Podakovanie

Abstrakt

Kľúčové slová

záverečná práca, šablóna, L^AT_EX, formátovanie textu, citácie

Abstract

Keywords

Final thesis, template, L^AT_EX, text formatting, citations

Obsah

Úvod	8
1 Súčasný stav	10
1.1 Znalostné grafy	10
1.1.1 Typy grafových databáz	11
1.1.2 Porovnanie s vektorovou a relačnou databázou	12
1.2 Klasické metódy extrakcie	13
1.2.1 Rozpoznávanie pomenovaných entít	13
1.2.2 Extrakcia vzťahov	14
1.3 Posun k využívaniu LLM	16
1.3.1 Generatívne a sémantické vlastnosti LLM	16
1.3.2 Schémové a bezschémové prístupy k extrakcii	17
1.3.3 Promptovacie techniky pri extrakcii znalostí	17
1.3.4 Motivácia využívania LLM v moderných IE systémoch	17
1.4 Existujúce prístupy pre tvorbu znalostných grafov z textu	18
1.5 Medze	20
1.5.1 Slovenský jazyk ako málo zastúpená doména	20
1.5.2 Právna doména ako špecializovaný kontext	21
1.5.3 Hybridné spracovanie textu a tabuliek	21
2 Návrh riešenia	22
2.1 Predspracovanie	22
2.1.1 Detekcia rozloženia	23
2.1.2 Extrakcia tabuliek	23
2.1.3 Extrkacia vzorcov	23
2.2 Trojkroková extrakcia	24
2.2.1 Detekcia v otvorenej doméne (ODD)	24
2.2.2 Úprava a spresnenie schémy (SR)	24
2.2.3 Schémou riadená extrakcia (SDE)	25
2.3 Vkladanie do databázy	26
3 Implementácia	29
3.1 Technologický stack	29
3.2 Kľúčové implementačné rozhodnutia	30
3.3 Prompt engineering pre slovenčinu	32
4 Experimenty a vyhodnotenie	34

4.1	Testovanie dáta a experimentálne prostredie	34
4.2	Priebeh extrakcie a kvantitatívne výsledky	34
4.3	Porovnanie s baseline	34
4.4	Obmedzenia a vyzvy	34
4.5	Diskusia a limitácie	34
Záver		35
Literatúra		36
	Použitie nástrojov umelej inteligencie	39

Zoznam značiek a skratiek

Úvod

v uvode obsiahnut problematiku

1 Súčasný stav

1.1 Znalostné grafy

Znalostné grafy (Knowledge Graphs, KG) predstavujú moderný spôsob reprezentácie a prepájania údajov, ktorý umožňuje strojom chápať vzťahy medzi entitami podobne, ako to robia ľudia.

Znalostný graf je grafová reprezentácia dát určená na zhromažďovanie a odovzdávanie vedomostí o reálnom svete. Skladá sa z dvoch základných prvkov. Uzly, ktoré predstavujú objekty zájmu (entity) a hrany, ktoré definujú vzťahy medzi týmito entitami [1].

Vynikajú v integrácii rôznorodých dát, kde znalostné grafy sú ideálne na spájanie informácií z mnohých dynamických a rozsiahlych zdrojov. Na rozdiel od tradičných databáz, grafy sú flexibilnejšie, kde schéma a dáta sa môžu voľnejšie rozvíjať. Podporujú navigačné operátory, ktoré dokážu rekurzívne vyhľadávať entity prepojené cestami ľubovoľnej dĺžky. Pomocou ontológií a pravidiel umožňujú strojom nielen ukladať fakty, ale o nich aj logicky uvažovať a odvodzovať znalosti. Najznámejšími verejne dostupnými znalostnými grafmi sú DBpedia, Freebase Wikidata, YAGO [1].

Znalostné grafy v poslednej dobe narástli na popularite ako štruktúrovaný spôsob reprezentácie faktov [2]. Generatívne veľké jazykové modely (LLM) sú náchylné na produkovanie halucinácií, ktoré pramenia najmä z medzier vo vedomostiach modelu, čím model generuje informácie nepodporené faktickými dátami a vytvára výstup, ktorý znie vierohodne, no je nepresný alebo úplne nesprávny [3, 4]. Ako reakciu na toto obmedzenie výskumníci navrhli rôzne stratégie augmentácie LLM externými znalosťami, pričom využitie znalostných grafov ako zdroja takýchto informácií preukázalo sľubné výsledky pri znižovaní halucinácií a zlepšovaní presnosti uvažovania [3–5].

V oblasti softvérového inžinierstva LLM excelujú v generovaní kódu, no zápasia s úlohami na úrovni celého repozitára, pretože nemajú prehľad o globálnej štruktúre projektu a majú tendenciu sa úzko zameriavať na konkrétne súbory, čo vedie k lokálnym chybám [6]. Existujúce prístupy zaobchádzajú s kódom ako s plochými dokumentami, čo nedokáže zachytiť zložité medzisúborové závislosti [6]. Tento problém možno zmierniť modelovaním repozitára ako znalostného grafu, kde uzly predstavujú entity kódu (súbory, triedy, funkcie) a hrany ich štruktúrne závislosti a vzťahy [7, 8]. Takáto reprezentácia umožňuje cieľené získavanie relevantného kontextu namiesto čítania nadbytočných súborov [6, 8].

Praktickým uplatnením znalostných grafov je aj diagnostika porúch priemyselných zariadení, kde súčiastky a stroje tvoria entity a hrany ich vzájomné vzťahy. Z textových opisov porúch zo senzorových sietí je možné postaviť doménovo špecifický znalostný graf zachytávajúci expertné znalosti o priemyselnom zariadení a následne pomocou LLM vykonávať kauzálnu analýzu na presnejšie určenie príčiny a lokalizácie poruchy [9].

Výrazné využitie nachádzajú znalostné grafy aj v kybernetickej bezpečnosti. CTI-Nexus automaticky konštruuje znalostné grafy z reportov o kybernetických hrozbách (Cyber Threat Intelligence, CTI) pomocou LLM extrakcie entít a vzťahov [10], zatiaľ čo KGV integruje LLM so štruktúrovanými znalostnými grafmi pre automatizované hodnotenie dôveryhodnosti CTI [11]. Systém LinkQ ukazuje, že nútením LLM dotazovať sa na znalostný graf pre fakty pri odpovedaní na otázky možno znížiť halucinácie aj v kritických doménach ako kyberbezpečnosť [5]. Podobné prínosy sú dokumentované v medicíne, kde integrácia medicínskych znalostných grafov do LLM zlepšuje diagnostické uvažovanie a znižuje riziko diagnostických chýb [12, 13], ako aj vo finančníctve, kde štruktúrovanie finančných dokumentov do znalostných grafov zlepšuje numerické uvažovanie a vysvetliteľnosť odpovedí LLM [14, 15].

Znalostné grafy poskytujú LLM faktické uzemnenie a explicitné znalostné cesty, čím znižujú halucinácie a zvyšujú presnosť uvažovania [2, 3]. Súčasne umožňujú získavať len relevantné podgrafy namiesto vkladania veľkých objemov textu do promptu, čo v aplikáciách ako finančné odporúčania preukázateľne znižuje veľkosť kontextového okna a zlepšuje cielenosť modelu na podstatné informácie [16].

1.1.1 Typy grafových databáz

Grafové databázy rozlišujeme na niekoľko typov, ktoré majú svoje špecifické určenia. Uvedené sú tri hlavné typy grafových databáz, ktoré sa najčastejšie vyskytujú pri znalostných grafoch:

- Orientovaný graf s označenými hranami (Directed Edge-Labelled Graph), je množina uzlov a orientovaných hrán, kde každá hrana má priradený jeden štítok (napríklad model RDF) [1].
- Heterogénny graf (Heterogeneous Graph), kde každému uzlu a hrane je priradený práve jeden typ [1].
- Graf s vlastnosťami (Property Graph) umožňuje priradovať štítky a páry vlastnosť-hodnota k uzlom aj hranám [1].

- Málo používaný avšak veľmi zaujímavý je hypergraf, ktorý umožňuje hranám spájať viac ako dve entity súčasne (komplexné hrany) [1].

V projekte je využívanou databázou na ukladanie dát grafová databáza s vlastnosťami (Property graphs), akou je napríklad Neo4j. Formálna definícia grafovej databázy s vlastnosťami podľa Hogan et al., ktorá uchováva dáta v grafe v podobe usporiadanej n -tice $G = (V, E, L, P, U, e, l, p)$. Platí: V je množina identifikátorov vrcholov (uzlov), E je množina identifikátorov hrán, L je množina typov, ktoré môžu niesť uzly aj hrany, P je množina mien vlastností, U je množina hodnôt vlastností, $e : E \rightarrow V \times V$ každú hranu priradí dvojici vrcholov (jej začiatočnému a koncovému uzlu), $l : V \cup E \rightarrow 2^L$ priraduje každému uzlu alebo hrane množinu typov, $p : V \cup E \rightarrow 2^{P \times U}$ priraduje každému uzlu alebo hrane množinu dvojíc (*vlastnosť, hodnota*). Intuitívne: vrcholy a hrany sú len identifikátory z univerzálnej množiny databázy, funkcia e špecifikuje štruktúru grafu (kto je s kým spojený), funkcie l a p nesú „sémantiku“, typy uzlov/hrán a ich atribúty.[1]

Tento typ vyniká, pretože ponúka vyššiu flexibilitu pri modelovaní komplexných vzťahov. Jeho hlavnou výhodou je, že umožňuje priamo anotovať hranu bez nutnosti vytvárania pomocných uzlov. Ak chceme napríklad k vzťahu medzi entitami priradiť rozširujúcu vlastnosť (napríklad v medicíne k hrane NEZIADUCI_UCINOK môžeme priradiť dodatočnú vlastnosť, aké konkrétne nežiadúce účinky spôsobil liek). Naopak v jednoduchých modeloch by sme museli vytvoriť samostatný uzol pre danú hranu [1].

V tejto práci modelujem vzťah medzi entitami v KG ako množinu trojíc (*triples*) v tvare (e_h, r, e_t) , kde e_h označuje začiatočnú entitu (*head*), e_t koncovú entitu (*tail*) a r vzťah medzi nimi. Takáto trojica zodpovedá orientovanej hrane $e_h \xrightarrow{r} e_t$ v orientovanom grafe, ktorý je podľa Hogan et al. [1] základným modelom pre znalostné grafy a je súčasne dátovým modelom štandardu RDF konzorcia W3C.

1.1.2 Porovnanie s vektorovou a relačnou databázou

V porovnaní s relačnými databázami sa dá jednoduchšie pristupovať k viac skokovým údajom alebo cestám, kde sa vyhýbajú relačným joinom. V relačnej databáze by sme každý typ uchovávali v jednotlivých tabuľkách, zatiaľ čo v grafovej databáze je to oddelené typom entity alebo vzťahu. Taktiež relačné databázy sa nevedia dynamicky prispôbiť novým typom a preto si vyžadujú vopred definovanú schému. Remodelovanie takejto databázy je veľmi nákladne a časovo náročné. K dátam v grafovej databáze môžeme ľahko pristupovať pomocou dopytovacieho jazyka Cypher, ktorý je súčasťou rodiny grafových dopytovacích jazykov spolu s SPARQL. Tieto jazyky podporujú okrem

štandardných relačných operátorov aj operátory, ktoré rekurzívne vyhľadávajú cestu medzi entitmi ľubovoľnej dĺžky, čo je v tradičnom SQL náročne vyjadriteľné [1].

Alternatívou by mohla byť vektorová databáza, kde sa údaje ukladajú ako vektory v multidimenzionálnom priestore. Dáta sa dajú následne vyhľadávať pomocou sémantickej podobnosti a vrátiť k najbližších susedov. Ich slabinou je strata explicitného relačného kontextu, ak otázka vyžaduje následovať reťaz $A \rightarrow B \rightarrow C$, čisto sémanticky podobné údaje môžu relevantnú väzbu minúť, pretože kľúčové údaje sú roztrúsené v dokumente [17]. Naopak znalostné grafy robia vzťahy explicitnými a umožňujú uvažovanie nad cestami, kde kľúčové uzly majú vzdialenosť n hrán (skokov). V moderných systémoch s využitím LLM sa technológie grafov a vektorov navzájom dopĺňajú [18].

1.2 Klasické metódy extrakcie

Extrahovanie informácií (Information Extraction, IE) alebo taktiež extrahovanie znalostí (Knowledge Extraction, KE) je úloha transformovania neštruktúrovaného textu prirodzeného jazyka na štruktúrovanú, strojovo čitateľnú reprezentáciu akými sú trojice entita vzťah. Klasické IE princípy rozdeľujú úlohu do podúloh, detekcia entít a ich prepojenie, riešenie koreferencie a extrahovanie vzťahov, ktoré sú spracované cieľovou schémou. Prvotné systémy, ako TextRunner a KnowItAll, sa spoliehali na špeciálne navrhnuté pravidlá alebo na pevne stanovené lingvistické vzory na extrakciu faktov z textu [19]. Tento prístup bol veľmi nákladný na expertnú prácu a zle sa prispôbovalo novým typom dokumentom [19]. Tieto limitácie motivovali posun k štatistickým a k neurónovým prístupom, kde vzory na extrahovanie informácií nie sú manuálne písané ale učenie z označených príkladov [20]. Súčasným trendom je vykonávať rozpoznávanie pomenovaných entít a extrakciu vzťahov súčasne v jednom kroku, čím sa znižuje riziko prenosu chýb medzi jednotlivými fázami spracovania [1].

1.2.1 Rozpoznávanie pomenovaných entít

Rozpoznávanie pomenovaných entít (Named Entity Recognition, NER) je dôležitá úloha v rámci spracovania prirodzeného jazyka, ktorej cieľom je v texte nájsť a zaradiť konkrétne názvy do preddefinovaných typov akými sú Osoba, Miesto, Organizácia alebo Vozidlo. V praxi je NER kľúčovým prvotným krokom pri budovaní znalostných grafov, pretože umožňuje z neštruktúrovaného textu extrahovať prvky, ktoré sa stanú uzlami v grafe.

Technológie NER prešli vývojom od jednoduchých pravidiel až po zložité neurónové siete:

1. Strojové učenie (štatistické metódy): Tieto metódy vnímajú rozpoznávanie entít ako úlohu sekvenčného značovania. Model prechádza text slovo po slove a rozhoduje, či dané slovo je začiatkom, vnútrom alebo koncom nejakej entity (často používaná BIES schéma, Beginning, Intermedia, Ending, Single) [19].
 - Najčastejšie využívanými štatistickými modelmi sú Markovove modely (HMM) alebo podmienené náhodné polia (CRF). Tieto modely sa učia štatistické závislosti medzi susednými slovami (napríklad že po slove študent pravdepodobne nasleduje meno osoby) [19].
 - Supervised learning vyžaduje veľké množstvo manuálne označených textov, aby sa model naučil správne vzory [19].
2. Hlboké učenie (neurónové siete): Moderné metódy využívajú hlboké neurónové siete, ktoré dokážu automaticky spracovať zložité súvislosti v texte bez nutnosti manuálneho navrhovania vzorov [19].
 - CNN (konvolučné neurónové siete) sa zamieravajú na lokálne vlastnosti v okolí slova na zachytávanie entít [19].
 - RNN (rekurentné neurónové siete) sú navrhnuté tak, aby lepšie spracovali kontextové vlastnosti v dlhých vetách. Avšak RNN môžu trpieť kontextovou zaujatosťou v neskorších prichádzajúcich slovách. Preto mnohé modely využívajú obojsmerné varianty (Bi-LSTM-CRF, GRU), ktoré čítajú text zľava aj sprava [19].

1.2.2 Extrakcia vzťahov

Extrakcia vzťahov (Relation Extraction, RE) je proces ako aj NER, ktorého cieľom je identifikovať a klasifikovať sémantické prepojenia medzi entitami, ktoré boli v texte nájdené. Zatiaľ čo NER nájde v texte jednotlivé inštancie entít, RE tieto entity spája do vzťahu, ktorý vysvetľuje ako spolu entity súvisia.

RE premieňa neštruktúrovaný text na štruktúrované znalosti, vo forme trojíc (e_h, r, e_t) . Model zistí či medzi dvoma entitami nejaký vzťah existuje, a ak existuje priradí im konkrétny typ vzťahu z preddefinovanej schémy. Taktiež môže sa využívať otvorená extrakcia, kde model nie je obmedzený a snaží sa extrahovať akýkoľvek vzťah priamo z gramatickej štruktúry vety [19].

Tieto prístupy sa používajú najmä na schématickú (preddefinovanú) extrakciu ale niektoré neurónové metódy sa dajú využiť aj pri otvorenej extrakcii:

1. Strojové učenie ako napríklad kernelové metódy, merali podobnosť medzi vetami pomocou matematických funkcií. Namiesto jednoduchých zoznamov slov analyzujú

celé syntaktické stromy vety, aby pochopili, ako sú slová gramaticky prepojené. Tieto metódy sa najčastejšie používajú pri schématickej extrakcii, kde je vopred daná konečná množina typov vzťahov. [19, 20]

2. Hlboké učenie:

- CNN sa zameriavajú na lokálny kontext v blízkosti entít [19].
- LSTM dokážu zachytiť dlhé závislosti v dlhých vetách a súvetiach [19].
- GCN (grafové neurónové siete) vnímajú vetu ako graf prepojených slov, čím im pomáha lepšie pochopiť zložité gramatické vzťahy [19].

Všetky tieto architektúry dostávajú na vstupe vetu (prípadne dokument) s označenými entitami a na výstupe priradujú dvojici entít typ vzťahu z preddefinovanej schémy. Niektoré varianty (encoder-decoder, Siamese siete) sa používajú aj pri otvorenej extrakcii vzťahov [19].

Okrem voľby architektúry sa modely líšia aj tým, v akom režime sa učia a aký typ dát spracúvajú. V praxi je často problémom nedostatok ručne anotovaných dát, prekrývajúc esa vzťahy v jednej vete alebo vzťahy vyjadrené naprieč viacerými vetami. Pre tieto situácie boli navrhnuté nasledujúce špeciálne prístupy RE:

- Distant supervised relation extraction využíva existujúcu znalostnú databázu na automatické označovanie tréningových viet. Ak KG obsahuje trojicu (e_h, r, e_t) , všetky vety s párom entít (e_h, e_t) dostanú typ r . Takto vzniknuté veľké množstvo automaticky označených dát umožňuje škálovateľný tréning klasických aj hlbokých modelov, no zároveň zanáša šum, pretože nie každá veta s daným párom entít daný vzťah skutočne vyjadruje. Moderné prístupy preto pridávajú špeciálne mechanizmy (PCNN, selective attention nad množinou viet, reinforcement learning či adversariálne učenie), ktoré sa snažia nespoľahlivé príklady filtrovať [19].
- Spoločná extrakcia entít a vzťahov (Joint NER and RE) sa spája do jedného modelu, čím odstraňuje šírenie chýb typické pre prístupy, kde chyby v NER kroku znehodnotia aj následnú extrakciu vzťahov. Medzi dva hlavné prístupy patrí zdieľanie vrstiev medzi úlohami (BiLSTM, BiGCN alebo GraphRel) a špeciálne anotovacie schémy (CasRel) [19].
- Few-shot a zero-shot extrakcia vzťahov rieši prípady, keď je málo alebo žiadne anotované dáta. Pri štandardnej konfigurácii dostane model K príkladov pre N typov vzťahov a má klasifikovať nové vety. Hlavné prístupy sú metric learning

- porovnávanie reprezentácií, meta-learning - model sa učí ako sa učiť" z malého množstva príkladov a doménová adaptácia - prenos znalostí medzi doménami [19].
- Extrakcia vzťahov na úrovni dokumentu rieši problém vzťahov vyjadrených naprieč viacerými vetami. Jednotlivé prístupy stavajú heterogénne grafy nad celým dokumentom, kde uzly reprezentujú spomenutia, vety a entity, a spracúvajú ich GCN (EoG, LSR, GAIN) [19].

Extrakcia vzťahov je nevyhnutná z niekoľkých kľúčových dôvodov. Pre budovanie znalostných grafov automatizuje extrémne náročnú manuálnu prácu pri vytváraní veľkých databáz vedomostí. Umožňuje vyhľadávateľovi odpovedať na priame otázky namiesto toho, aby len zobrazili zoznam stránok. Špecializované odbory ako medicína kde RE pomáha vedcom identifikovať interakcie medzi liekmi alebo proteínmi v tisíckach odborných článkoch, čo urýchľuje výskum chorôb, alebo právo kde sa používa na analýzu súdnych rozhodnutí, vyhľadávanie argumentov v spisoch alebo automatické spracovanie zmlúv. [19–21]

1.3 Posun k využívaniu LLM

Posun v oblasti extrahovania informácií alebo znalostí smerom k využitiu veľkých jazykových modelov predstavuje zásadnú zmenu paradigmy od tradičných štatistických a pravidlových metód k generatívnej a jazykovo riadenej štruktúre. Ako bolo spomenuté, tradičné metódy sa spoliehali na fragmentáciu procesov, ktoré boli obmedzené potrebou expertného návrhu a veľkého množstva označených dát [18, 19].

1.3.1 Generatívne a sémantické vlastnosti LLM

LLM transformujú proces extrakcie tým, že fungujú ako kognitívne mechanizmy, ktoré dokážu priamo syntetizovať štruktúrované reprezentácie z neštruktúrovaného textu. Na rozdiel od klasických metód, ktoré hľadali faktické vzory pomocou definovaných pravidiel alebo zhľukovania, LLM využívajú svoje schopnosti sémantického zjednotenia a generatívneho modelovania [18, 19].

Hlavnými charakteristikami tohto prístupu sú:

- Generatívne modelovanie znalostí, kde LLM dokážu vytvárať štruktúrované dáta priamo z prirodzeného jazyka [18].
- Sémantické zjednotenie, integrácia heterogénnych zdrojov vedomostí prostredníctvom uzemnenia v prirodzenom jazyku [18].

- Promptami riadená orchestrácia, koordinácia zložitých pracovných postupov budovania znalostných grafov pomocou interakcie založenej na promptoch [18].

1.3.2 Schémové a bezschémové prístupy k extrakcii

Využitie LLM pre extrakciu prebieha v dvoch hlavných paradigmách. Prvou je metóda založená na schéme, ktoré sa spoliehajú na explicitnú schému znalosti, ktorá poskytuje štrukturálne vedenie a sémantické obmedzenia. LLM tu slúžia ako asistenti, ktorí dopĺňajú inštanície do vopred definovaných ontológií. Existujú aj adaptívne schémy, ktoré sa dynamicky vyvíjajú počas procesu extrakcie bez nutnosti opätovného tréningu modelu. Druhou metódou je metóda bez schémy. Cieľom je získavať znalosti priamo bez závislosti na vopred definovaných schémach. Tieto metódy uprednostňujú flexibilitu a otvorené objavovanie [18].

1.3.3 Promptovacie techniky pri extrakcii znalostí

Kľúčové techniky pre LLM modely na funkciu extrakcie:

- In-context learning (ICL) je schopnosť modelov riešiť extrakčné úlohy bez explicitného tréningu, len na základe abstraktného popisu (Zero-Shot learning) alebo niekoľkých príkladov (Few-Shot learning) v rámci vstupu [22].
- Chain-of-Thought (CoT) prompting, inštruovanie LLM, aby pri extrakcii postupoval krok za krokom, čo zlepšuje logickú súvislosť a organizáciu znalostí [22].
- Multi-agentové systémy, ktoré nasádzajú špecializovaných agentov, ktorí spolupracujú na extrakcii, normalizácii entít a riešení konfliktov [18].

1.3.4 Motivácia využívania LLM v moderných IE systémoch

V súčasnosti sa systémy na extrahovanie informácií čoraz viac orientujú na využitie veľkých jazykových modelov, ktoré môžu automatizovať celý proces extrakcie alebo sa kombinovať s metódami strojového a hlbokého učenia. Prečo sa prechádza k LLM riešeniam? Hlavné dôvody tohto posunu súvisia s prekonávaním obmedzení tradičných metód:

- Škálovateľnosť a dátová riedkosť. Tradičné systémy založené na pravidlách často zlyhávajú pri prechode medzi doménami a vyžadujú obrovské množstvo tréningových dát [18]. LLM dokážu vďaka vedomostiam z predtréningu definovať extrakčné ciele pre nové oblasti s minimálnym počtom príkladov [22].

- Rýchla adaptácia domény. LLM umožňujú výskumníkovi ponúkať prispôsobiteľné služby špecifické pre danú doménu (právo, informatika, biológia, medicína, ...) bez nutnosti náročného manuálneho označovania dát [22].
- Riešenie fragmentácie procesu. Pri klasických postupoch viedlo oddelulé spracovanie (napríklad najprv NER a potom RE) k postupnému prenosu a kumulácii chýb [18, 19]. LLM umožňujú zjednotené generatívne rámce, ktoré tieto kroky integrujú do jedného procesu [18].
- Spracovanie zložitého kontextu. Tradičné modely na úrovni viet často nedokázali zachytiť vzťahy rozložené naprieč celými dokumentmi (príklad právo). LLM majú schopnosť spracovávať dlhé texty (napríklad dlhé vedecké články alebo právne zmluvy) a chápať zložitú logickú a sémantickú súvislosť [19, 21, 22]. LLM využívajú mechanizmus seba-pozornosti (self-attention), ktorý im umožňuje rozlišovať význam slov podľa kontextu, napríklad či slovo „Tesla“ označuje vedca, automobilku alebo automobil [21].
- Zníženie závislosti na expertoch. Hoci experti sú stále dôležití na overovanie, LLM dramaticky znižujú potrebu manuálneho navrhovania ontológií a pravidiel, čo urýchľuje budovanie vedomostných systémov [18].

1.4 Existujúce prístupy pre tvorbu znalostných grafov z textu

V súčasnom výskume extrakcie informácií a konštrukcie znalostných grafov dominuje niekoľko kľúčových frameworkov a modelov, ktoré posúvajú hranice automatizácie, presnosti a škálovateľnosti.

V oblasti komplexných vedeckých a multimodálnych dát vyniká toolkit DeepKE, ktorý podporuje extrakciu na úrovni dokumentov, multimodálne scenáre akými sú text a obraz, a učenie s nízkym množstvom dát. DeepKE integruje rôzne architektúry ako CNN, GCN alebo trnasformery a pre multimodálne úlohy implementuje model IFAformer, ktorý prepája textové a vizuálne vlastnosti [23].

Pre globálnu optimálnu extrakciu grafu z jednej vety je určený framework OneIE, ktorý využíva spoločný neurónový model a beam dekodér na zachytenie vzájomných závislostí medzi entitami a udalosťami. Na druhej strane, model KG-BERT sa zameriava na dopĺňanie znalostných grafov tým, že s entitami a vzťahmi narába ako s textovými sekvenciami. To mu umožňuje využiť lingvistický kontext z predtrénovaných váh BERT modelu na predikciu chýbajúcich faktov [24, 25]

Pre autonómnú konštrukciu grafov v obrovskom meradle bol vyvinutý framework AutoSchemaKG, ktorý produkuje rodinu grafov ATLAS (Automated Triple Linking And Schema induction). Tento systém spracováva desiatky miliónov dokumentov z webových korpusev bez potreby vopred definovaných schém, pričom modeluje nielen entity, ale aj udalosti, ktoré sú nevyhnutnou reprezentáciou znalostného grafu. Výsledkom je graf s miliardami hrán, ktorý dosahuje 95% sémantickej podobnosti autonómne vytvorenej schémy s manuálne vytvorenou schémou človekom, a taktiež výrazne zlepšuje multi-hop vyhľadávanie a uvažovanie LLM [26].

Ak je cieľom extrakcia s existujúcou schémou (napríklad Wikidata), framework EDC (Extract, Define, Canonicalize) ponúka trojfázové riešenie. EDC najskôr vykoná otvorenú extrakciu, následne vygeneruje definície pre extrahované prvky. Nakoniec využije Schema Retriever na identifikáciu a extrakciu relevantných častí cieľovej schémy [27].

Pre náročné produkčné prostredie spoločnosť Apple vyvinula systém ODKE+ (Open Domain Knowledge Extraction), ktorý sa špecializuje na udržiavanie čerstvosti znalostných grafov. Využíva modulárnu pipeline s modulmi akými sú Grounder na elimináciu halucinácií a Corroborator na obodovanie a normalizáciu faktov, vďaka čomu dosahuje presnosť až 98,8% [28].

Pre špecifickú doménu e-commerce bol vytvorený multi-agentový framework Auto-PKG. Automaticky indukuje typy produktov a extrahuje atribúty z multimodálneho obsahu ako text a obrázky. Kľúčovým prvkom je agent KGD (Knowledge Graph Decision), ktorý slúži ako jediné rozhranie pre zápis a robí rozhodnutia o zlúčení alebo nahradení uzlov na udržanie globálnej konzistencie [29].

Vedeckú akvizíciu znalostí rieši agentický framework KnoBuilder, ktorý strategicky plánuje dopyty na vyplnenie medzier v znalostiach a využíva vyvíjajúci sa graf ako svoju externú štruktúrovanú pamäť [30].

Pre špecializované expertné domény, bol navrhnutý framework SAC-KG, ktorý využíva LLM ako doménových expertov v procese s tromi komponentami akými sú Generator, Verifier a Pruner. Verifikátor detekuje chyby pomocou repozitára RuleHub s viac ako 7000 pravidlami, zatiaľ čo Pruner určuje smer rastu grafu, čo umožňuje budovať grafy o veľkosti miliónov vrcholov s presnosťou cez 89% [31].

Ďalším frameworkom, ktorý zavádza kontrolu úrovni kvality je GraphJudge. Využíva uzavretý LLM GPT-4o-mini na očistenie textu a generovanie kandidátnych trojíc, zatiaľ čo jemne doladený otvorený model LLaMA-2-7B funguje ako sudca grafu, ktorý tieto trojice verifikuje voči dokumentu [32].

Výskum v oblasti bezpečnostných domén tiež navrhuje Knowledge Routing Network (KRN) riadiaci hierarchické moduly LoRA pre prispôsobenie modelov k špecifickej terminológii pri zachovaní efektivity [33]

Framework spoločnosti SAP nahrádza drahé LLM volania pri konštrukcii grafu priemyselnými knižnicami NLP (ako SpaCy) využívajúcimi závislostnú gramatiku, pričom si zachováva 94% výkonu LLM riešení pri výrazne nižších nákladoch [34].

1.5 Medze

Hoci sa oblasť spracovania právnych textov pomocou NLP a LLM v posledných rokoch dynamicky rozvíja, väčšina existujúcich prác sa sústreďuje na jazykovo a doménovo odlišný kontext, než aký predstavuje slovenské finančné právo. Nedávny prehľadový článok pokrývajúci 131 štúdií v oblasti právneho NLP konštatuje, že táto oblasť čelí špecifickým výzvam vyplývajúcim z dĺžky dokumentov, komplexnosti právneho jazyka a obmedzenej dostupnosti otvorených právnych datasetov. Bližšia analýza však odhaľuje tri konkrétne medzery, ktoré priamo motivujú návrh predkladaného systému [21].

1.5.1 Slovenský jazyk ako málo zastúpená doména

Existujúce datasety a modely pokrývajú predovšetkým angličtinu, čínštinu, nemčinu, francúzštinu a v menšej miere ďalšie rozšírené jazyky. Multilingválne korpusy ako MultiLegalPile pokrývajú 24 jazykov naprieč 17 jurisdikciami, no viaceré jazyky vrátane menších európskych zostávajú výrazne nedostatočne pokryté. Slovenčina v tomto zozname prakticky absentuje, čo znemožňuje priame využitie predtrénovaných právnych modelov typu legal-bert alebo Lawformer. Autori prehľadu zároveň explicitne upozorňujú, že rozšírenie multilingválnych schopností do menej zastúpených jazykov je jednou z otvorených výskumných výziev, pretože jednotlivé právne systémy majú vlastné termíny a štandardy dokumentov, ktoré sa medzi jazykmi výrazne líšia. Slovenský právny jazyk navyše obsahuje špecifickú morfológiu, diakritiku a terminológiu (napr. označenia ako paragraf, odsek, písmeno), ktoré nemožno spoľahlivo preniesť z anglického modelu [21].

Na extrakciu pomenovaných entít by sme mohli využiť nástroje ako SpaCy alebo modely typu SlovakBERT, kde by sa modeli pretrénovali na právnu doménu. Kým pri extrakcii pomenovaných entít môže postačovať menšie množstvo anotovaných príkladov, napríklad dataset WikiGoldSK [35], extrakcia vzťahov medzi entitami je náročnejšia a vyžaduje rozsiahlejšie a konzistentne anotované dáta. V právnej doméne je takáto anotácia obzvlášť časovo náročná, pretože si vyžaduje jazykové aj odborné porozumenie dokumentom. Použitie veľkých jazykových modelov preto predstavuje praktickú alter-

natívu ku klasickému dotrénovaniu modelov, keďže ich správanie možno do istej miery riadiť promptmi a inštrukciami bez potreby rozsiahleho anotovaného datasetu.

1.5.2 Právna doména ako špecializovaný kontext

Aj v rámci samotného právneho NLP sa väčšina výskumu sústreďuje na úlohy ako klasifikácia, sumarizácia alebo predikcia rozhodnutí súdu, zatiaľ čo konštrukcia znalostných grafov z právnych textov ostáva málo preskúmaná. Využitie ontológií a znalostných grafov v právnej doméne je relatívne zriedkavé, hoci má značný potenciál posilniť usudzovanie nad zložitými právnymi textami. Ich nasadenie však komplikuje pojem právnej domény, kedy sa význam právnych pojmov vyvíja v čase. Pre oblasť slovenského finančného práva — ktoré sa vyznačuje hustou sieťou krížových odkazov medzi predpismi, paragrafmi a odsekmi, neexistuje verejne dostupná schéma ani anotovaný dataset, ktorý by takéto vzťahy explicitne zachytával (existujúci dataset WikiGoldSK pre NER nemá široké zastúpenie práva [35]). To znamená, že schéma uzlov a vzťahov (napr. PravnyPredpis, Paragraf, OBSAHUJE, ODKAZUJE_NA) musí byť navrhnutá osobitne, s rešpektom k formálnej štruktúre slovenskej legislatívy a k danému typu dokumentu [21].

1.5.3 Hybridné spracovanie textu a tabuliek

Tretia, prakticky najpodstatnejšia medzera spočíva v tom, že existujúce riešenia pre konštrukciu znalostných grafov pracujú takmer výlučne s plynulým textom. Slovenské zákony z oblasti finančného práva (napr. zákony upravujúce kolektívne investovanie, dane či účtovníctvo) však obsahujú významnú časť normatívneho obsahu v podobe tabuliek — sadzieb, limitov, kategórií majetku alebo výpočtových koeficientov. Predspracovanie právnych textov je samo o sebe náročnou úlohou, pretože dokumenty majú zložité vnorené štruktúry, klauzuly v klauzulách a krížové odkazy na iné prípady, predpisy alebo ustanovenia, čo sťažuje ich segmentáciu na koherentné jednotky pre analýzu. Tabuľky túto komplexnosť ešte zvyšujú, pretože strácajú zmysel pri transformácii do čistého textu a vyžadujú vlastný spôsob extrakcie entít a vzťahov, ktorý zachová riadkovo-stĺpcovú sémantiku a zároveň ju prepojí s textovým kontextom paragrafu, v ktorom sa tabuľka nachádza. Táto medzera vytvára priestor pre návrh hybridného prístupu, ktorý dokáže kombinovať extrakciu informácií z textových aj tabuľkových častí dokumentu, a integroval by výsledky do jedného konzistentného grafu.

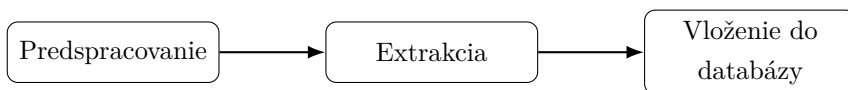
2 Návrh riešenia

Cieľom riešenia je transformovať PDF dokument slovenského právneho predpisu z neštruktúrovaného textu na štruktúrovaný znalostný graf (KG), ktorý je schopný zodpovedať na vecné otázky o obsahu zákona a zachovať odkaz na pôvodný text. Vstupom je PDF zbierky zákonov, výstupom je Neo4j property graf doplnený o tri vektorové indexy nad uzlami, chunkami a vzťahmi.

Generický prístupom by bolo naivné prevedenie PDF na text a extrakcia trojíc jedným LLM volaním. Na slovenských právnych dokumentoch zlyháva z troch dôvodov: tabuľky a vzorce sa pri čistej textovej konverzii rozsypú alebo úplne stratia, slovenská diakritika spôsobuje rozpad rovnakej entity na viacero uzlov (problematika LLM spracovania), a bez vopred známej schémy LLM produkuje nekonzistentné typy entít a vzťahov naprieč chunkami. Návrh riešenia preto stojí na štyroch princípoch:

- hybridné spracovanie - text, tabuľky aj vzorce sa extrahujú samostatnými trasami
- schéma odvodená z dát a nie ručne definovaná - lógia sa najprv objaví, potom zjednotí, potom vynúti
- jazyková normalizácia - jednotná ASCII reprezentácia ID a mien typov
- zachovanie pôvodu - každá entita je viazaná na chunk a chunk na dokument, takže ku každému tvrdeniu v grafe existuje stopa do pôvodného textu

Spracovanie je súvislá trojkroková postupnosť: predspracovanie (detekcia rozloženia strán, samostatná extrakcia tabuliek a vzorcov, oddelenie zvyšného textu), extrakcia (ODD $\rightarrow$ SR $\rightarrow$ SDE), a vloženie do databázy (MERGING uzlov a hrán cez normalizované ID a vybudovanie vektorových indexov). Tento tok dát je znázornený na obrázku č.1.



Obr. 1: Graf popisujúci spracovanie.

2.1 Predspracovanie

Čisté spracovanie PDF dokumentu nie je dostačujúce pre legislatívne dokumenty. Napríklad tabuľky, ktoré sú nositeľmi dôležitých, opisných a rozvíjajúcich vlastností. Dokonca, sa táto tabuľka rozkladá na viacero strán. Pri čítaní tabuľky, ako reťazca by sa tieto informácie roztrúsili a zničil by sa kontext celej tabuľky. Vzorce sa taktiež

vyskytujú naprieč dokumentom a v PDF dokumente su uchované ako obrázky - nástroj na čítanie textu z dokumentu by tento obrázok nerozpoznal a nenašiel.

2.1.1 Detekcia rozloženia

V tomto kroku sa každá strana dokumentu konvertuje na obrázok s rozlíšením 200 DPI. Aplikuje sa DocLayout-YOLO model, ktorý prejde a zdetekuje rozloženie každej strany. Tento detektor je nakonfigurovaný aby rozoznal päť cieľových tried: tabuľka, obrázok, diagram, izolovaný vzorec a popis vzorca. Detekcie s istotou pod 0,3 sú zahodené. Pre každú nájdenú detekciu, naväzujúci box je vystrihnutý z obrázku stránky a uložený ako bezstratový PNG súbor. Všetky susedné obrázky na základe strany (ak rozdiel strán je menší ako 2) sú zoskupené, aby tabuľky, ktoré sa rozliehajú na viacero strán, boli pri sebe ako jeden logický celok.

2.1.2 Extrakcia tabuliek

Každá skupina tabuliek je spracovaná v dvoch krokoch. Prvý krok, OCR schopný GPT 5.4 mini model, prijme postupne po 2 orezaných obrázkoch skupiny spoločne so system promptom, ktorý nariaďuje aby bol výstup tabuľka v podobe HTML. Tento prompt zakazuje sumarizáciu tabuľky a nariaďuje aby model zachoval doslovný text bunky.

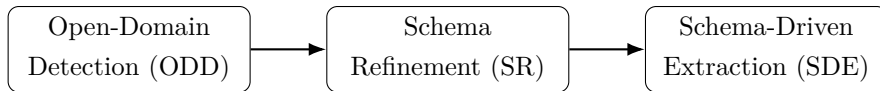
Druhý krok, výsledný HTML výstup je poskytnutý ďalšiemu LLM volaniu, kde tento krát ma model za úlohu transformovať tabuľku do hierarchickej podoby vrcholov a vzťahov: tabuľka je označená ako koreňový vrchol, a list ako každý riadok tabuľky (uchováva informácie z jednotlivých stĺpcov riadka vo vlastnostiach, kde kľúč je hlavička stĺpca a hodnota je bunka v riadku a stĺpci). Je to cesta ako zachovať význam tabuľky v znalostnom grafe. Vnútorne strany viac-stranovej tabuľky sú vyradené zo zoznamu strán PDF dokumentu, pre neskoršiu extrakciu aby sa predišlo duplicitám rovnakej informácie.

2.1.3 Extrakcia vzorcov

Detektor rozloženia okrem tabuliek označí aj izolovaný vzorec a popis vzorca. Každá detekcia vzorca je orezaná z obrázku strany a uložená ako PNG. Tieto obrázky sú následne posielané modelu GPT 5.4 mini, ktorý je system promptom inštruovaný aby vrátil doslovný LaTeX prepis vzorca bez ďalšieho komentára. Úloha je úzko vymedzená na OCR matematického zápisu a nevyžaduje široké uvažovanie. Výsledné LaTeX reťazce sú pripojené ku grafovým dokumentom ako vlastnosti príslušných uzlov, aby sa zachovali ako strojovo spracovateľný obsah, nie ako neprečítateľný obrázok.

2.2 Trojkroková extrakcia

Po dokončení predspracovania, zvyšné strany dokumentu sú rozsekané na chunky textov a spracované cez trojkorovú extrakciu inšpirovanú výskumami ODKE+, AutoSchemaKG, EDC [26–28]. Táto extrakcia sa skladá z troch postupných krokov: detekcia v otvorenej doméne (open-domain detection, ODD) - objav typov, úprava schémy (schema refinement, SR) - normalizácia, a v poslednom rade je to schémov riadená extrakcia (schema-driven extraction, SDE) - vynútená extrakcia inštancií. Tento tok je zobrazený na obrázku č. 2.



Obr. 2: Extrakcia inšpirovaná ODKE+ [28]

2.2.1 Detekcia v otvorenej doméne (ODD)

Cieľom tohto prvého kroku je nájsť, aké typy entít a vzťahov sa v dokumente vyskytujú. Text je rozdelený pomocou nástroja RecursiveCharacterTextSplitter, s veľkosťou chunku 1200 charakterov a prekryvom 200 charakterov. Relatívne väčšie okno textu je zvolené schválne. V tejto etape model GPT-5.4-mini je vynútený aby vyhľadal typy a nie konkrétne inštancie. Širšie kontextové okno napomáha aby model videl viac ako len jednu vetu včase, čo je veľmi dôležité v legislačných dokumentoch, kde napríklad slovo „základ dane“ je uvedené niekoľko viet predtým ako je použité.

Každý chunk c_i je paralelne posiadaný LLM agentovi, ktorého system prompt inštruuje aby našiel a vrátil všeobecné ale rozlíšiteľné typy entít a typy vzťahov. Agentovi je explicitne napísané aby sa zameriaval na typy ako sú zákony, paragrafy, práva a povinnosti. Všetky diakritické znaky sú normalizované do ASCII aby znalostný graf používal jednotnú slovenskú výslovnosť bez diakritiky. Výstup tohto kroku je zjednotený zoznam schém chunkov: $S_{ODD} = \bigcup_{i=1}^{|C|} S_i$. Schému môžeme zobrazit ako $S_i = (T_V^{(i)}, T_E^{(i)})$, kde $T_V^{(i)}$ sú typy entít (vrcholov v znalostnom grafe) a $T_E^{(i)}$ sú typy vzťahov (hrán v znalostnom grafe) medzi entitami.

2.2.2 Úprava a spresnenie schémy (SR)

Zjednotené schémy z predošlého kroku detekcie (ODD) S_{ODD} , sú poslané na úpravu schémy modelu, spolu s existujúcou schémou znalostného grafu: $S^* = \Phi(S_{ODD}, S_O)$. Schéma S_O je zvolená podľa podmienky, či existuje už nejaká definovaná schéma v znalostnom grafe, a ak nie zadefinuje sa ako pevne daná schéma, ktorá je konštantná

naprieč finančným právom. Pri prvotnom vytváraní KG je tak model usmernený, aby vytvoril dostatočne hustú právnú typológiu na následnú extrakciu.

Tento krok úpravy ma čistú lingvistickú úlohu: rieši duplikáty spôsobené dikaritikou alebo vynechaním písmena (Dan a Daň alebo ClenskyStat a ClenskStat) a v neposlednom rade zjednocuje sémanticky podobné typy a synonymá (Doklad, DanovyDoklad a HodnoveryDoklad zjednotí do Doklad). Taktiež sa tým rieši koherentnosť, aby nebolo viacero typov toho istého významu. Zároveň nedovoľuje aby sa spojili sémanticky odlišné typu ako Dokument a Osoba. A v neposlednom rade je to generalizácia, kde príliš špecifické typy sa nahradia všeobecnejším typom, ak to neznižuje právnú zrozumiteľnosť.

User a system prompt explicitne nariaďujú modelu reťazové uvažovanie (Chain of Thought), aby zreteľne identifikoval duplicity spôsobené preklepmi alebo diakritikou, porovnal surové dáta so základnou ontológiou a skontroloval zlučovanie nesúvisiacich kategórii. Navyše vynucuje pre typy vzťahov aby boli písané pomocou UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE) a pre typy entít Pascal-Case. Toto explicitné rozdelenie zlepšuje kvalitu deduplikácie najmä pre vzťahy, model najprv "premýšľa" mapovanie a až potom ho serializuje. Z týchto dôvodov je volaný väčší model Gemini 2.5 Pro, ktorý podporuje hlbšie uvažovanie/odvodzovanie. Taktiež sa uchováva merge log, ktorý mapuje všetky typy do akého typu entity alebo vzťahu boli zjednotené. Pomáha to pri debuggovaní alebo auditovaní.

2.2.3 Schémou riadená extrakcia (SDE)

Posledný krok je extrakcia pomocou natvrdo nanútenej schémy (jednotný krok NER a RE), v tomto prípade je to upravená S^* schéma. Text je v tomto kroku znovu rozdelený na menšie chunky s veľkosťou 1024 charakterov a prekryvom 128 charakterov. Tento menší chunk je podnietený odlišnou úlohou ako pri ODD. V tomto kroku GPT-5.4-mini model musí extrahovať konkrétne inštancie entít a vzťahov medzi nimi. Menšia veľkosť textu vynucuje model k extrahovaniu presnejších a lepšie uzemnených trojíc (e_h, r, e_t) , kde e_h je začiatková entita vzťahu, e_t je koncová entita vzťahu a r je vzťah medzi danými entitami. Menšie kontextové okno znižuje pravdepodobnosť, že sa dva nesúvisiace fakty zmiešajú do jedného vzťahu. Chunky sú posielané paralelne s limitovanou súbežnosťou a pause-and-retry mechanizmom, v prípade vyčerpania užívateľových tokenov za minútu.

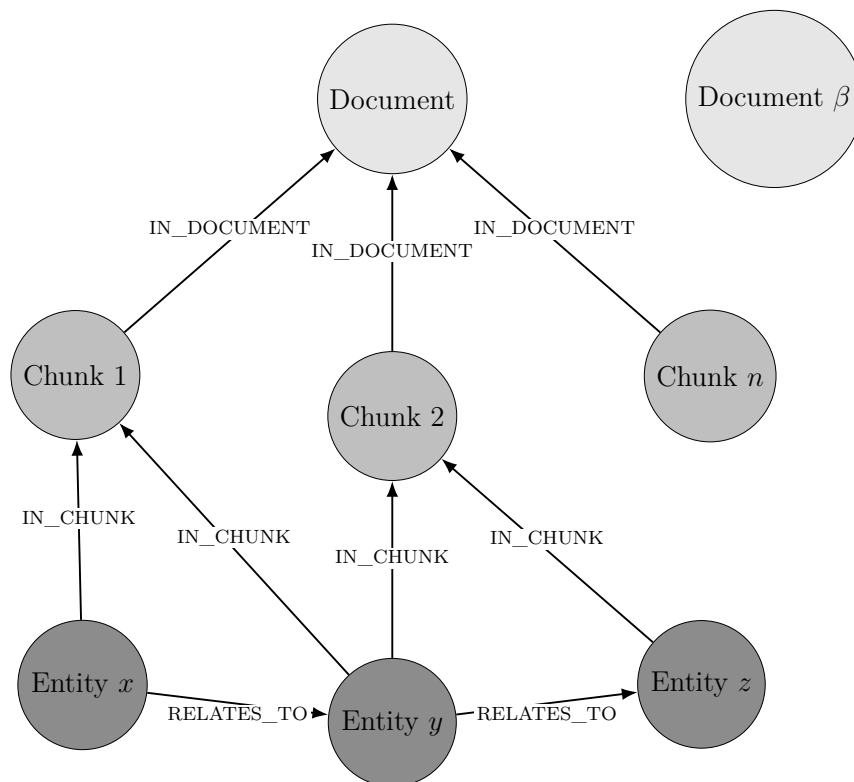
Aby striktné dodržanie schémy S^* nevedlo k strate kontextu, system prompt zavádza špeciálnu konvenciu. Ak je typ vzťahu v S^* príliš všeobecný (napr. MA_NAROK_NA), model uloží konkrétne spresnenie do vlastnosti vzťahu po kľúčom **add** (napr. add: vratenie dane). Týmto sa rieši napätie medzi striktnou schémou a expresivitou textu. Typ vzťahu zostáva normalizovaný a vyhľadateľný v grafe, ale doslovný význam sa zachováva ako stopa pre neskoršie vyhľadávanie.

Prompt nariaďuje zopár kvalitatívnych pravidiel. Po prvé, pravidlo najšpecifickejšieho typu, ak schéma obsahuje aj všeobecnejšie aj špecifickejšie pasujúce typy, použije sa špecifickejší. Po druhé, pravidlo closest-or-skip, ak v schéme nie je presne pasujúci typ, model použije najbližší existujúci, alebo entitu úplne preskočí (radšej nič ako halucinovaný typ). Po tretie, vlastnosti len ak sú explicitné, model nesmie vymýšľať vlastnosti uzlov ani vzťahov, môže uložiť iba to, čo je v texte priamo uvedené. A v neposlednom rade, id uzla musí byť označené v neurčitku, rieši sa tým koherentnosť ale aj jednoduchšie zjednocovanie v databáze.

Pre každý chunk c_i , model vracia uzly a vzťahy, ktoré korešpondujú upravenej schéme S^* . Týmto krokom získame GraphDocument $G_i = (V_i, E_i, src_i)$. Následne ľahká kontrola a opracovanie, znormalizuje id uzlov do title case a pre vzťahy do UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE), a vynechá trojice ktorým chýba zdroj alebo cieľ. Extrahované uzly z chunku sú spojené s uzlom Chunk pomocou vzťahu IN_CHUNK. Neskôr vďaka tomuto vzťahu v kroku na získavanie dát, môžeme získať originálny text, ktorý podporuje dané trojice.

2.3 Vkládanie do databázy

Extrahované grafové dokumenty (GraphDocument) sú vo finálnom kroku vložené do Neo4j grafovej databázy. Uzly sú vytvorené pomocou MERGE nad ich normalizovaným ID (konkrétna inštancia entity napr. Colna Dan), tak že ich opakované zmienenia medzi chunkami sú zjednotené do jednotnej inštancie. Vzťahy u taktiež vytvárané pomocou MERGE logiky aby sa predišlo duplicitným hranám. Narozdiel od pridaných extrahovaných uzlov a Chunk uzlov, je v štruktúre databázy pridaný uzol typu Document, ktorý nesie ID názvu dokumentu. V právnych dokumentoch sú to ID ako ZZ_2003_595_20260101, kde ZZ je zbierka zákona, 2003 rok, 595 číslo zákona a 20260101 dátum poslednej zmeny dokumentu. Všetky chunky z tohto dokumentu sú pospájané pomocou vzťahu IN_DOCUMENT. Výslednú štruktúru a hierarchiu databázy môžeme vidieť na obrázku 3. Výhodou tejto architektúry je zachovanie vzťahu pôvodu odkiaľ je daná entita prevzatá. Taktiež je veľmi jednoduché zistiť pri spracovaní viacerých dokumentoch, v akom dokumente/zákone sa vyskytuje.



Obr. 3: Štruktúra znalostného grafu: dokument, chunk, entita and vzťah. Obrázok prevzatý z blogu Neo4j [36] a upravený na moje riešenie.

Aby sa chunky z rôznych dokumentov vzájomne nezjednotili do jedného uzla, ID každého chunku má tvar `chunk_<i>_<document_id>` (`chunk_42_ZZ_2003_595_20260101`). Tým sa zaručí, že MERGE nad chunkami zachová oddelenosť chunkov medzi dokumentmi, aj keď ich poradové číslo i je rovnaké.

Vstupom tohto kroku nie sú iba grafové dokumenty z SDE, ale aj grafové dokumenty extrahované z tabuliek a LaTeX prepisy vzorcov z fázy predspracovania. Všetky tri zdroje sa zlúčia do jedného zoznamu a vložia naraz, aby výsledný graf jednotne reprezentoval text aj nelineárne prvky dokumentu (tabuľky a vzorce sa stávajú uzlami a vzťahmi v rovnakom grafe ako entity z textu).

Každý extrahovaný uzol dostane okrem svojho doménového typu (FyzickaOsoba, Paragraf, DanovaSadzba) aj pomocný spoločný typ `__Entity__`. Toto je dôsledok parametra `baseEntityLabel=True` pri volaní vkladania do grafovej databázy. Spoločný typ slúži ako jednotný typ pre neskorší vektorový index entít, bez ktorého by sa musel index budovať osobitne nad každým doménovým typom, pričom typov je v S^* rádovo desiatky. Hneď po vložení sa však typ `__Entity__` explicitne odstráni z uzlov Chunk a Document, aby do entitno-vektorového indexu nepretiekli pomocné štrukturálne uzly a nezašumili tak RAG.

Po vložení uzlov a hrán sa nad grafom vybudujú tri samostatné vektorové indexy pomocou Neo4jVector: nad textom uzlov Chunk (pre vyhľadanie pôvodného textového kontextu k otázke), nad ID a vlastnosťami entít (pre nájdenie uzemnenej entity v otázke), nad typmi a kontextom vzťahov (pre výber relevantného typu hrany pri iteratívnej traverzácii). Embeddingy sú generované modelom OpenAI text-embedding-3-large. Rozdelenie na tri oddelené indexy umožňuje cielené zrýchlenejšie hybridné vyhľadávanie.

3 Implementácia

3.1 Technologický stack

Python

Python slúži ako orchestračný jazyk celého systému. Je zvolený pre svoju rozsiahlu podporu v oblasti LLM. Jeho dynamické typovanie a množstvo knižníc umožňujú rýchlu integráciu komponentov - od klasifikácie obrázkov cez veľké jazykové modely až po databázove ovládače. V kóde sa využíva aj jeho podpora typových anotácií a dátových tried.

LangChain

LangChain je knižnica navrhnutá na budovanie systémov okolo LLM. Poskytuje abstrakcie nad rôznymi poskytovateľmi jazykových modelov, čo znamená, že výmena jedného modelu za druhý si nevyžaduje prepísanie aplikačnej logiky. V projekte sa využíva najmä jeho schopnosť reťaziť operácie, spravovať kontextové okná, pracovať s nástrojmi a budovať agentické workflowy.

Neo4j

Neo4j je grafová databáza postavená na modeli grafov s vlastnosťami. Entity v grafe reprezentujú uzly s vlastnosťami, menovkami a vzťahy sú typové hrany. Tento typ služby som zvolil kvôli dobrému softvérovému riešeniu, kde sa dá spravovať databáza, vizualizovať dáta a narábať s nimi pomocou Cypher queries. Taktiež je tu veľmi dobré riešenie prepájania grafov s vektorovým úložiskom, kde rôzne uzly môžu byť vektorizované a prepojené pomocou ich unikátneho ID.

LLM modely

Projekt využíva viac poskytovateľov LLM modelov súčasne. Modely sú volené od ich schopnosti, či to je rýchlosť, cena, kvalita spracovania, reasoning a ich určenie. Sú to konkrétne modely GPT-5.4-mini od OpenAI a Gemini 2.5 Pro od Googlu. Modely sú ukázané v metrike cena vstup/výstup, tokenov za minútu a požiadaviek za minútu.

- GPT-5.4-mini: 0.75\$/4.50\$, 200 000 TPM a 500 RPM
- Gemini 2.5 Pro: 1.25\$/10.00\$, 2 000 000 TPM a 150 RPM

Správy a inštrukcie sa následne posielajú v dvoch vstupoch. Prvým je systémová správa (SystemMessage, system_prompt), ktorá definuje rolu, kontext a pravidlá,

ktorými sa model má riadiť. Druhou je ľudská správa (HumanMessage, user role), ktorá obsahuje samotný vstup používateľa.

DocLayout-YOLO

DocLayout-YOLO je špecializovaná neurónová sieť založená na architektúre YOLO, ktorá bola natrénovaná na úlohy detekcie prvkov v dokumentoch. Na rozdiel od všeobecných detektorov objektov, ktoré hľadajú autá alebo ľudí, tento model rozpoznáva dokumentové prvky ako sú odseky textu, nadpisy, tabuľky, obrázky, vzorce, popisy obrázkov a ďalšie. Výsledkom je štruktúrovaná reprezentácia vizuálneho rozloženia stránky s orezanými obdĺžnikmi a ich pravdepodobnosťami. Na to aby sa tento nástroj dal použiť potrebujeme konverziu dokumentu v PDF formáte na obrázky. To zaručí knižnica pdf2image, ktorá premení PDF dokument na rastrové obrázky.

3.2 Kľúčové implementačné rozhodnutia

Mechanizmus semaforu

Pri spracovávaní dokumentu pomocou LLM modelu, ktoré sú rozdelené na desiatky či i stovky chunkov, nastávajú 2 problémy. Prvým problémom je, keď posielame veľa požiadaviek paralelne, tak prťažíme API a vyčerpáme limit tokenov za minútu (TPM - Tokens Per Minute). Druhým je, keď jedna požiadavka zlyhá kvôli týmto limitám, zvyšné požiadavky s istotou zlyhajú taktiež. Tieto problémy riešim pomocou koordinačného mechanizmu.

Inicializácia začína vytvorením 3 kontrolných prvkov. Prvým je semafor, ktorý riadi konkurenčný limit, koľko úloh naraz môže byť obsluhovaných. V tomto implementovaní je to 5 konkurenčných úloh. Zabraňuje to systému aby neodoslal stovky požiadaviek naraz. Druhým prvkom je globálny prepínač *pause switch*, ktorý začína v zapnutej pozícii. Pokiaľ tento prepínač je v zapnutej pozícii, systém môže voľne posilať API požiadavky. Akonáhle prepínač sa dostane do pozície vypnutej, každá pracovná úloha v systéme sa zastaví na rovnakom synchronizačnom bode a čaká. Posledným prvkom je zámok, ktorý povoľuje v prípade chyby alebo zlyhania pracovať iba na jednej úlohe.

Po nastavení týchto riadiacich prvkov, je každému chunku pridelená asynchrónna úloha. Každá úloha sa riadi rovnakou rutinou a má povolené až tri pokusy na dokončenie svojho procesu. Na začiatku každého pokusu úloha naprv overí stav globálneho prepínača. Ak je prepínač vypnutý, úloha čaká, kým sa znova nezapne.

Keď je prepínač aktívny, úloha sa pokúsi získať miesto v semafore. Akonáhle miesto získa, spustí samotnú extrakcie alebo detekciu volaním LLM modelu. Ak požiadavka prebehne úspešne, úloha skončí a vráti svoj výsledok. Ak volanie zlyhá vyhodnotením

výnimky a úloha má ešte k dispozícii ďalšie pokusy, nastáva situácia, kedy sa úloha pokúsi získať zámok. Ten zaručuje, že iba úplne prvá úloha, ktorá zlyhanie zaregistruje, sa stáva zodpovednou za jeho riešenie. Táto úloha následne vypne globálny prepínač, čím okamžite pozastaví všetky ostatné úlohy v systéme. Následne sa systém uspí na 60 sekúnd, čo je dostatočne dlhý čas na to, aby sa rate limit modelu obnovil, a následne sa prepínač opäť zapne.

Ak pracovná úloha nakoniec vyčerpá všetky tri pokusy bez úspechu, vzdá sa a propaguje výnimku ďalej, namiesto toho aby blokovala zvyšok systému. Medzitým boli všetky úlohy spustené v rovnakom okamihu, skrz mechanizmus task creation a počas celého procesu bežia paralelne v rámci jedného event loopu. Hlavná rutina čaká pomocou agregačnej operácie gather, kým každá jednotlivá úloha neskončí.

Precíznosť tohto návrhu spočíva v spôsobe, akým zaobchádza so zlyhaním ako so zdieľaným signálom. Jediné zlyhanie je interpretované ako dôkaz toho, že celý spracovateľský systém je obmedzený, a preto je cooldown aplikovaný globálne a nie individuálne. Zámok, tu zohráva zásadnú rolu, pretože bez neho by mohli desiatky úloh detekovať to isté zlyhanie v rovnakom okamihu a spustiť svoj 60 sekundový interval nezávisle.

Štruktúrovaný výstup

V kontexte spracovania prirodzeného jazyka prostredníctvom LLM predstavuje štruktúrovaný výstup (a. j. structured output) zásadnú techniku, ktorá zaručuje, že odpoveď volania modelu bude zodpovedať definovanej schéme. Vďaka tomu sa eliminuje chybovosť, ktorú zapríčiňuje parsovanie výstupných dát z modelu, zaručí plynulosť spracovania a zdieľania dát vo workflowe. Dáta môžeme jednoducho deserializovať do Python objektov (dictionary, dataclass alebo Pydantic modelu). V prostredí knižnice LangChain existujú dva odlišné spôsoby, ktoré majú svoje špecifické vlastnosti, výhody a obmedzenia v závislosti od použitého modelu.

Prvý prístup, ktorý sa v kóde uplatňuje predovšetkým na modely od spoločnosti OpenAI, je stratégia poskytovateľa (ProviderStrategy). Tento prístup využívajú modely, ktoré natívne podporujú štruktúrovaný výstup (OpenAI, Anthropic, xAI, Gemini)[37]. Je to principiálny prístup pri komunikácii s modelom skrz agentovu vrstvu. Schéma očakavaného výstupu sa pri tomto prístupe odovzdáva ako parameter formátu odpovede (response_format) priamo pri vytváraní agenta.

Hlavnou výhodou tejto stratégie je deterministická záruka, že vrátený objekt bude validný voči zadanej schéme. Ďalším významným prínosom je integrácia s agentovým ekosystémom, čo umožňuje rozšírenie o nástroje, kontextovú pamäť a iteratívne uvažo-

vanie bez nutnosti meniť spôsob, akým sa štruktúrovaný výstup spracováva. V kóde projektu sa tento prístup uplatňuje pri kľúčových úlohach, akými sú detekcia v otvorenej doméne a schémou riadená extrakcia.

Druhý prístup, použitý v projekte pri komunikácii s modelmy Gemini, je metóda priameho štruktúrovaného výstupu (`with_structured_output`). V kóde sa tento princíp kombinuje s metódou založenou na prevode schémy do formátu JSON Schema. Tento prístup pracuje na úrovni samotného modelu bez agentovej vrstvy. Metóda vracia nový Runnable, ktorý pri každom volaní vloží schému do natívneho parametra API (`response_format` pre OpenAI a pre Gemini `response_mime_type` alebo `response_json_schema`) a automaticky parsuje odpoveď[38, 39]. Ak je schéma Pydentic trieda, tak požiadavka vráti inštanciu tejto triedy, a pri JSON Schema zas vráti dict.

Výhodou tohto prístupu je jeho jednoduchosť volania a priamy návrat výsledku bez potreby extrahovania z vnoreného kontextového výstupu. V kóde sa tento prístup uplatňuje pri spracovávaní tabuliek do HTML, transformácií HTML tabuliek do grafovej reprezentácie a pri úprave schémy.

3.3 Prompt engineering pre slovenčinu

Dôraz na ASCII v názvoch typov (ako constraint pre property graph labels), chain-of-thought pokyny, few-shot príklady, riešenie skloňovania.

Kvalita spracovania zadania modelu závisí nielen od schopností zvoleného modelu, ale aj od formy, v akej je modelu zadaná inštrukcia. Prompty sa musia zamerať na tieto triedy:

- Štruktúra promptu - modely reagujú výrazne presnejšie ak je vstup členený do sekcií. Prompt je písaný v Markdown formáte, kde nadpisy a podnápisy sú označené pomocou # alebo ##. Typické sekcie promptu bývajú: ÚLOHA, KONTEXT, VSTUP, POKYNY a PRÍKLADY.
- Konkrétnosť inštrukcie - inštrukcie musia byť presné a jazykovo presne definované. Ak model nemá explicitne napísané čo a ako má spraviť, výstup úlohy nemusí zodpovedať našim požiadavkam.
- Pravidlá a príklady - model musí mať presne ohraničené čo môže. Ak model explicitne neohraničíme pravidlami, model nemusí dodržiavať inštrukcie. Taktiež pár príkladov ako ma úlohu vykonať dopomôže kvalite spracovania (few-shot learning).

- Špecifiká pre slovenčinu - modelom je vynútené ako dany typ vzťahu alebo entity majú spracovať. Entity alebo uzly v grafe sú označované ako Title Case, typy entít PascalCase a vzťahy UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE). Dané triedy musia byť spracované bez diakritiky. Keby nie sú normalizované na nediakritické slová mohlo by sa stať že by daný extrahovaný typ by bol s diakritikou, v neskoršom spracovaní ďalšieho chunku bez diakritiky, a vznikli by duplicitné údaje.

Príklad systémovej inštrukcie na úpravu schémy pre model Gemini 2.5 Pro:

```
# ROLA
Si expertný ontológ a dátový inžinier špecializujúci sa na znalostné grafy.
Tvojou úlohou je vyčistiť, normalizovať a deduplikovať schému entít a vzťahov.

# CIELE
1. **Deduplikácia a sémantické zjednotenie**: Zlúč synonymá a sémanticky blízke typy do jedného kanonického typu.
2. **Normalizácia formátu**:
  - Uzly: PascalCase (napr. DanovePriznanie).
  - Vzťahy: UPPER_SNAKE_CASE (napr. MA_POVINNOST).
  - **STRIKTNÉ PRAVIDLO**: Odstráň všetku diakritiku (á->a, č->c, atď.) zo všetkých názvov.
3. **Ontologická integrita (KRITICKÉ)**:
  - NIKDY nezlučuj entity z rôznych kategórií: Aktér (Osoba) vs. Proces/Dokument (Ziadost),
    Miesto (Kraj) vs. Politický útvar (Stat).
  - NIKDY nezlučuj protichodné vzťahy (VSTUPUJE_DO vs. VYSTUPUJE_Z).
4. **Generalizácia**: Príliš špecifické uzly nahraď všeobecnejšími (napr. "Sluha" -> "Osoba"), ak
  to neznižuje zrozumiteľnosť právneho/biznisového kontextu.

# PRAVIDLÁ PRE ZÁKLADNÚ ONTOLÓGIU
- Ak je poskytnutá Základná ontológia, má prednosť v pomenovaní.
- Ak surový typ (Open Domain) zodpovedá typu v základnej ontológii, mapuj ho naň.
- Ak je surový typ unikátny a dôležitý, pridaj ho ako nový typ.
```

4 Experimenty a vyhodnotenie

9 s.

4.1 Testovanie dáta a experimentálne prostredie

1 s. Opis vybraných právnych textov (konkrétny zákon alebo colný sadzobník), štatistika vstupu (strany, tabuľky, odhadované entity), hardvér, použité verzie modelov.

4.2 Priebeh extrakcie a kvantitatívne výsledky

3 s.

Metriky z tvojho pipeline:

- Počet chunkov po oboch chunkovaniach - ODD: počet detegovaných typov uzlov a relácií per chunk, histogram - Refinement: kompresia schémy (napr. 150 surových typov → 45 kanonických), merge log s ukážkami ("Dan" + "Daň" + "dan" → "Dan") - SDE: počet extrahovaných inštancií uzlov a relácií - Tabuľková vetva: počet detegovaných tabuliek vs. manuálne spočítané, úspešnosť prevodu - Čas behu jednotlivých fáz, počet LLM volaní, odhad nákladov

4.3 Porovnanie s baseline

1 s. Napr. LLMGraphTransformer bez schema refinement kroku — ukáže pridanú hodnotu tvojho trojfázového návrhu. Alebo porovnanie s ODD-only (bez refinementu).

4.4 Obmedzenia a vyzvy

4.5 Diskusia a limitácie

zaver 1 s. Čo sa ukázalo ako silné stránky (konsolidácia schémy, hybridné spracovanie tabuliek), kde sú slabé miesta (náklady na LLM, závislosť na kvalite PDF, viacstránkové tabuľky s rozdielnou štruktúrou).

Záver

Literatúra

1. HOGAN, A. et al. Knowledge Graphs. *ACM Computing Surveys*. 2021, **54**(4), 1–37. ISSN 1557-7341. Dostupné z DOI: 10.1145/3447772.
2. LAVRINOVICS, E.; BISWAS, R.; BJERVA, J.; HOSE, K. *Knowledge Graphs, Large Language Models, and Hallucinations: An NLP Perspective*. 2024. Dostupné z arXiv: 2411.14258 [cs.CL].
3. AGRAWAL, G.; KUMARAGE, T.; ALGHAMDI, Z.; LIU, H. *Can Knowledge Graphs Reduce Hallucinations in LLMs? : A Survey*. 2024. Dostupné z arXiv: 2311.07914 [cs.CL].
4. PUSCH, L.; CONRAD, T. O. F. *Combining LLMs and Knowledge Graphs to Reduce Hallucinations in Question Answering*. 2025. Dostupné z arXiv: 2409.04181 [cs.CL].
5. LI, H.; APPLEBY, G.; ALPERIN, K.; GOMEZ, S. R.; SUH, A. *Mitigating LLM Hallucinations with Knowledge Graphs: A Case Study*. 2025. Dostupné z arXiv: 2504.12422 [cs.HC].
6. OUYANG, S. et al. *RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph*. 2025. Dostupné z arXiv: 2410.14684 [cs.SE].
7. ATHALE, M.; VADDINA, V. Knowledge Graph Based Repository-Level Code Generation. In: *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, 2025, s. 169–176. Dostupné z DOI: 10.1109/llm4code66737.2025.00026.
8. YANG, B. et al. *Enhancing repository-level software repair via repository-aware knowledge graphs*. 2025. Dostupné z arXiv: 2503.21710 [cs.SE].
9. XIE, X.; WANG, J.; HAN, Y.; LI, W. Knowledge Graph-Based In-Context Learning for Advanced Fault Diagnosis in Sensor Networks. *Sensors*. 2024, **24**(24). ISSN 1424-8220. Dostupné z DOI: 10.3390/s24248086.
10. CHENG, Y.; BAJABER, O.; TSEGAI, S. A.; SONG, D.; GAO, P. *CTINexus: Automatic Cyber Threat Intelligence Knowledge Graph Construction Using Large Language Models*. 2025. Dostupné z arXiv: 2410.21060 [cs.CR].
11. WU, Z.; TANG, F.; ZHAO, M.; LI, Y. *KGV: Integrating Large Language Models with Knowledge Graphs for Cyber Threat Intelligence Credibility Assessment*. 2025. Dostupné z arXiv: 2408.08088 [cs.CR].

12. GAO, Y. et al. Leveraging Medical Knowledge Graphs Into Large Language Models for Diagnosis Prediction: Design and Application Study. *JMIR AI*. 2025, 4, e58670. ISSN 2817-1705. Dostupné z DOI: 10.2196/58670.
13. JIA, M.; DUAN, J.; SONG, Y.; WANG, J. *medIKAL: Integrating Knowledge Graphs as Assistants of LLMs for Enhanced Clinical Diagnosis on EMRs*. 2025. Dostupné z arXiv: 2406.14326 [cs.CL].
14. MISHRA, A.; ANIL, A. *Structure First, Reason Next: Enhancing a Large Language Model using Knowledge Graph for Numerical Reasoning in Financial Documents*. 2026. Dostupné z arXiv: 2601.07754 [cs.CL].
15. LEE, C.; JEONG, Y.; SHIN, J.; KIM, H.; KIM, J. *Knowledge Graph Construction for Stock Markets with LLM-Based Explainable Reasoning*. 2025. Dostupné z arXiv: 2601.11528 [cs.DB].
16. SPADEA, F.; SENEVIRATNE, O. *Parallel and Multi-Stage Knowledge Graph Retrieval for Behaviorally Aligned Financial Asset Recommendations*. 2025. Dostupné z arXiv: 2511.11583 [cs.LG].
17. MAYO, M. *Vector Databases vs. Graph RAG for Agent Memory: When to Use Which*. [online]. MachineLearningMastery, 2026 [cit. 2026-04-26]. Dostupné z: <https://machinelearningmastery.com/vector-databases-vs-graph-rag-for-agent-memory-when-to-use-which/>.
18. BIAN, H. *LLM-empowered knowledge graph construction: A survey*. 2025. Dostupné z arXiv: 2510.20345 [cs.AI].
19. ZHONG, L.; WU, J.; LI, Q.; PENG, H.; WU, X. *A Comprehensive Survey on Automatic Knowledge Graph Construction*. 2023. Dostupné z arXiv: 2302.05019 [cs.IR].
20. MONCECCHI, G.; MINEL, J.-L.; WONSEVER, D. A survey of kernel methods for relation extraction. In: *Workshop on NLP and Web-based technologies*. Bahía Blanca, Argentina, 2010. Dostupné tiež z: <https://shs.hal.science/halshs-00532988>.
21. ARIAI, F.; MACKENZIE, J.; DEMARTINI, G. Natural Language Processing for the Legal Domain: A Survey of Tasks, Datasets, Models, and Challenges. *ACM Computing Surveys*. 2025, 58(6), 1–37. ISSN 1557-7341. Dostupné z DOI: 10.1145/3777009.

22. ATEIA, S.; KRUSCHWITZ, U.; SCHOLZ, M.; KOSCHMIDER, A.; ALMOHAISHI, M. LLM-Based Information Extraction to Support Scientific Literature Research and Publication Workflows. In: *New Trends in Theory and Practice of Digital Libraries*. Springer Nature Switzerland, 2025, s. 90–99. ISBN 9783032061362. ISSN 1865-0937. Dostupné z DOI: 10.1007/978-3-032-06136-2_9.
23. ZHANG, N. et al. *DeepKE: A Deep Learning Based Knowledge Extraction Toolkit for Knowledge Base Population*. 2023. Dostupné z arXiv: 2201.03335 [cs.CL].
24. YAO, L.; MAO, C.; LUO, Y. *KG-BERT: BERT for Knowledge Graph Completion*. 2019. Dostupné z arXiv: 1909.03193 [cs.CL].
25. LIN, Y.; JI, H.; HUANG, F.; WU, L. A Joint Neural Model for Information Extraction with Global Features. In: JURAFSKY, D.; CHAI, J.; SCHLUTER, N.; TETREAULT, J. (ed.). *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, 2020, s. 7999–8009. Dostupné z DOI: 10.18653/v1/2020.acl-main.713.
26. BAI, J. et al. *AutoSchemaKG: Autonomous Knowledge Graph Construction through Dynamic Schema Induction from Web-Scale Corpora*. 2025. Dostupné z arXiv: 2505.23628 [cs.CL].
27. ZHANG, B.; SOH, H. *Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction*. 2024. Dostupné z arXiv: 2404.03868 [cs.CL].
28. KHORSHIDI, S. et al. *ODKE+: Ontology-Guided Open-Domain Knowledge Extraction with LLMs*. 2025. Dostupné z arXiv: 2509.04696 [cs.CL].
29. HONGWIMOL, P. et al. *AutoPKG: An Automated Framework for Dynamic E-commerce Product-Attribute Knowledge Graph Construction*. 2026. Dostupné z arXiv: 2604.16950 [cs.AI].
30. MENA, B. J. S. [Regular] KnoBuilder: An LLM-Agent for Autonomous and Personalized Knowledge Graph Construction from Unstructured Text. In: *NORA: The First Workshop on Knowledge Graphs & Agentic Systems Interplay*. 2025. Dostupné tiež z: <https://openreview.net/forum?id=teewCPCv2m>.
31. CHEN, H. et al. *SAC-KG: Exploiting Large Language Models as Skilled Automatic Constructors for Domain Knowledge Graphs*. 2024. Dostupné z arXiv: 2410.02811 [cs.AI].

32. HUANG, H.; CHEN, C.; SHENG, Z.; LI, Y.; ZHANG, W. Can LLMs be Good Graph Judge for Knowledge Graph Construction? In: CHRISTODOULOPOULOS, C.; CHAKRABORTY, T.; ROSE, C.; PENG, V. (ed.). *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, 2025, s. 10929–10948. ISBN 979-8-89176-332-6. Dostupné z DOI: 10.18653/v1/2025.emnlp-main.554.
33. PENG, L.; YANG, P.; JUEXIANG, Y. et al. The construction and refined extraction techniques of knowledge graph based on large language models. *Scientific Reports*. 2026, **16**, 8104. Dostupné z DOI: 10.1038/s41598-026-38066-w.
34. MIN, C. et al. *Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale*. 2025. Dostupné z arXiv: 2507.03226 [cs.AI].
35. ŠUBA, D.; ŠUPPA, M.; KUBÍK, J.; HAMERLIK, E.; TAKÁČ, M. *WikiGoldSK: Annotated Dataset, Baselines and Few-Shot Learning Experiments for Slovak Named Entity Recognition*. 2023. Dostupné z arXiv: 2304.04026 [cs.CL].
36. KOHLWEY, E. *GraphRAG field guide: Navigating the world of advanced RAG patterns* [online]. Neo4j Developer Blog, 2024 [cit. 2026-04-20]. Dostupné z: <https://neo4j.com/blog/developer/graphrag-field-guide-rag-patterns/>.
37. LANGCHAIN, INC. *Structured output, LangChain documentation* [online]. LangChain, Inc. [cit. 2026-04-22]. Dostupné z: <https://docs.langchain.com/oss/python/langchain/structured-output>.
38. *ChatGoogleGenerativeAI reference* [online]. [cit. 2026-04-22]. Dostupné z: https://reference.langchain.com/python/langchain-google-genai/chat_models/ChatGoogleGenerativeAI/response_schema.
39. *LangChain OpenAI reference* [online]. [cit. 2026-04-22]. Dostupné z: https://reference.langchain.com/python/langchain-openai/chat_models/base/BaseChatOpenAI/with_structured_output.

Použitie nástrojov umelej inteligencie